

SQL injection with filter bypass via XML encoding

Platform	PortSwigger
Date	2026-09-1
Difficulty	Medium
Category	Web
Time Taken	20 minutes
Link	https://portswigger.net/web-security/sql-injection/lab-sql-injection-with-filter-bypass-via-xml-encoding
Tools Used	Burb Suite, Hackvertor extension

Lab Objective

perform a SQL injection attack to retrieve the admin user's credentials, then log in to their account.

Steps to Solve

Step 1: Recon

Opened the store page, selected an item, and clicked "Check stock" to trigger the target request.

Step 2: Intercept and send to Repeater

Captured **POST /product/stock** in Burp Suite. The request body is XML, with productId and storeId parameters. Sent it to Repeater for manipulation.

Step 3: Test for injection, hit the WAF

Injected a UNION-based payload into storeId:

```
1 union select username || '~' || password from users
```

Response: 403 Forbidden "Attack detected". The WAF blocks the request based on plaintext keywords (union, select, from) in the raw body.

Step 4: Research an encoding bypass

Since the request body is XML, looked up numeric character references for union, select, and from decimal entities like **u** for u, **s** for s, **f** for f. XML allows numeric references for any character, not just the five reserved ones, so this is valid XML that decodes back to plaintext after parsing.

Step 5: First encoded attempt: WAF bypassed, but query broke

Result: 200 OK, but response was 0 units. The WAF was bypassed successfully but in retyping the payload I switched from `||` concatenation to a comma-separated column list, which doesn't match the single-column output the original query expects. The failure was a query-structure mismatch, not an encoding problem.

Step 6: Hackvertor for reliable encoding

Installed the Hackvertor extension (per the lab hint) and wrapped the original, correct payload in an `<@hex_entities>` tag:

```
<@hex_entities>union select username || '~' || password from users</@hex_entities>
```

This encoded the entire clause at once (letters and structure intact) rather than hand-editing individual characters.

Step 7: Success

Response: 200 OK, body returned administrator~<password> alongside the normal wiener and carlos stock rows. Used the extracted password to log into /my-account as administrator. Lab marked Solved.

Screenshots

https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net/product?productId=1



Description:

We offer a completely unique gift wrapping experience - the gift that just keeps on giving. We can crochet any shape and size to order. We also collect worldwide, we do the hard work so you don't have to.

The gift is no longer the only surprise. Your friends and family will be delighted at our bespoke wrapping, each item 100% original, something that will be talked about for many years to come.

Due to the intricacy of this service, you must allow 3 months for your order to be completed. So, organization is paramount, no leaving shopping until the last minute if you want to take advantage of this fabulously wonderful new way to present your gifts.

Get in touch, tell us what you need to be wrapped, and we can give you an estimate within 24 hours. Let your funky originality extend to all areas of your life. We love every project we work on, so don't delay, give us a call today.

London

Check stock

964 units

[Return to list](#)

Site map

Scope

Issues

Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

phingest.portswigger.com

0a0c002603f4b17f809e3f21009200db.web-se

www.youtube.com

	Host	Method	URL	Params	Status code	Length	MIME type	Title	Notes	Time requested
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/academyLabHeader		101	147				22:04:30 1 Se...
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/		200	10869	HTML	SQL injection with filter ...		22:01:27 1 Se...
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	POST	/product/stock		200	115	text			22:04:33 1 Se...
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/product?productId=1		200	5019	HTML	SQL injection with filter ...		22:04:29 1 Se...
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/image/productcatalog/p...							
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/image/productcatalog/p...							
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/image/productcatalog/p...							
	https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net	GET	/image/productcatalog/p...							

Request

Response

Inspector

Notes

Discover

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

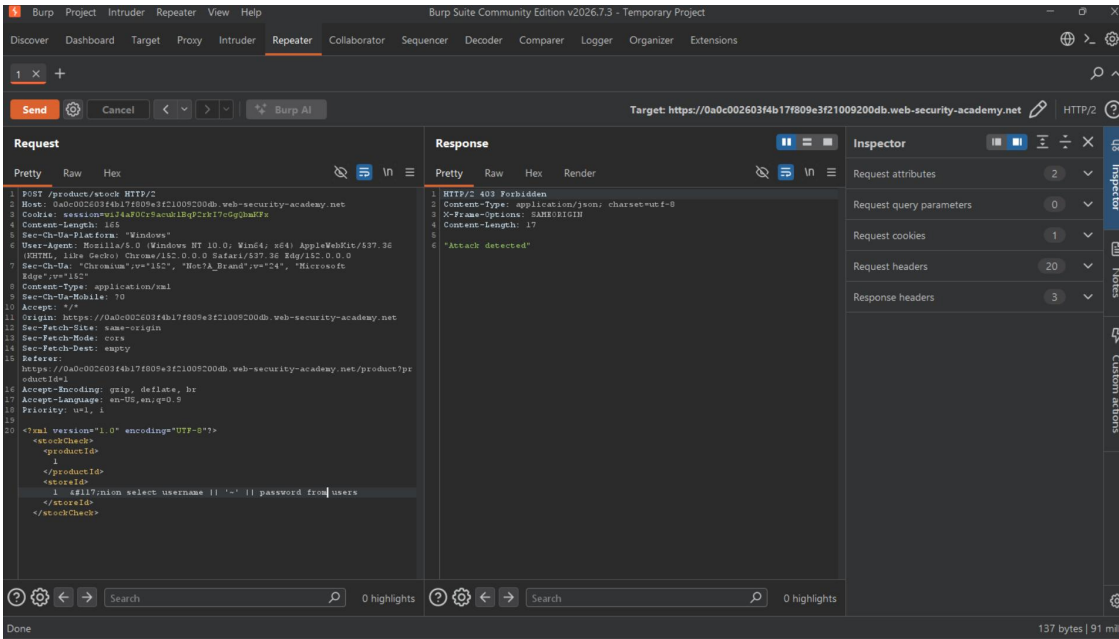
Target: https://0a0c002603f4b17f809e3f21009200db.web-security-academy.net HTTP/2

Request

Response

Inspector

Notes



XML escape sequence to encode the u

ALL SEARCH IMAGES VIDEOS MAPS COPILOT MORE

In XML, you can encode the lowercase letter **u** in several ways, depending on whether you want to escape it as a character entity or as a numeric code. Since **u** is **not** a reserved XML character (like **<**, **>**, **&**, **"**, **'**), you normally **don't need to escape it at all**. However, if you explicitly want to encode it:

Options to encode **u** in XML

Method	Example	Notes
Decimal numeric entity	<code>&#117;</code>	117 is the Unicode code point for u .
Hexadecimal numeric entity	<code>&#x75;</code>	0x75 is the hex code for u .
Direct character	<code>u</code>	Safe to use directly in XML if not part of markup.

Example XML snippet

XML escape sequence to encode the s letter

ALL SEARCH IMAGES VIDEOS MAPS COPILOT MORE

However, if you **still want to encode it** (for example, to represent it as a numeric or named entity), you can use:

- **Decimal numeric entity:** `s`
- **Hexadecimal numeric entity:** `s`

Example:



XML escape sequence to encode the f letter

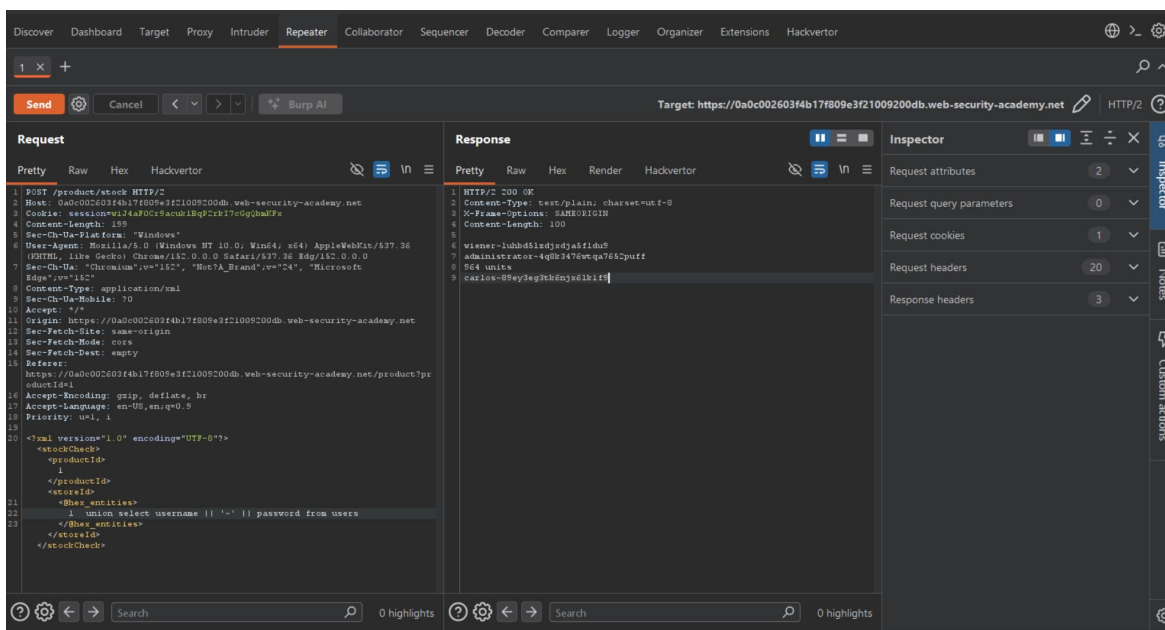
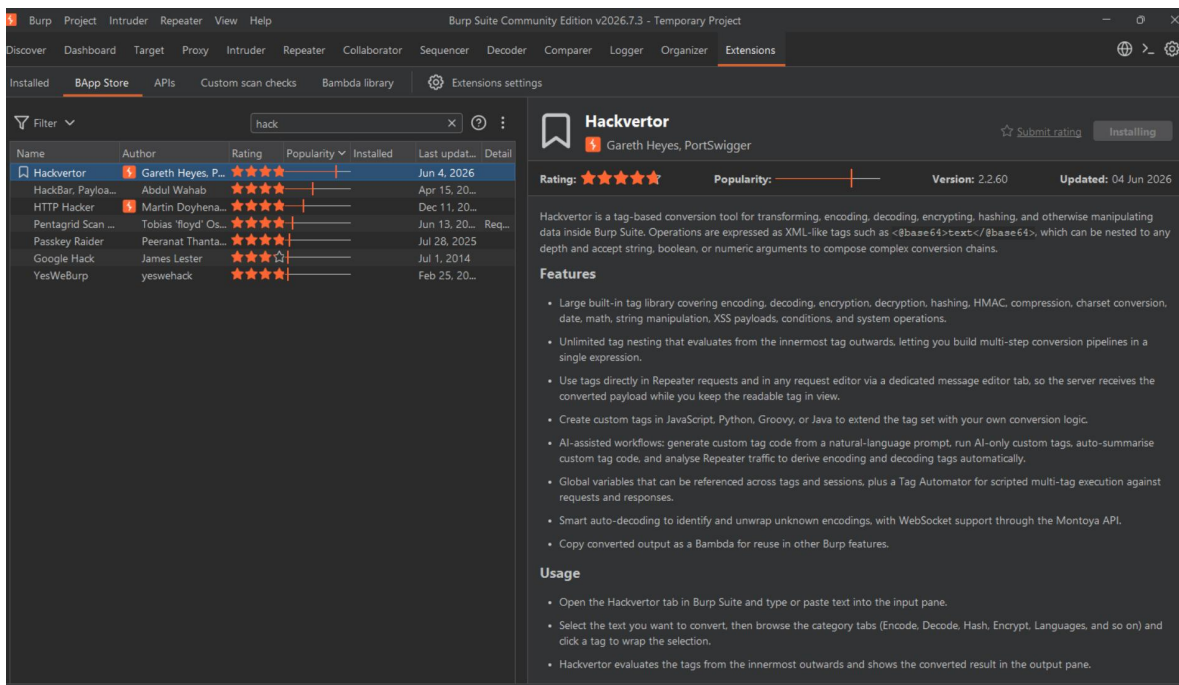
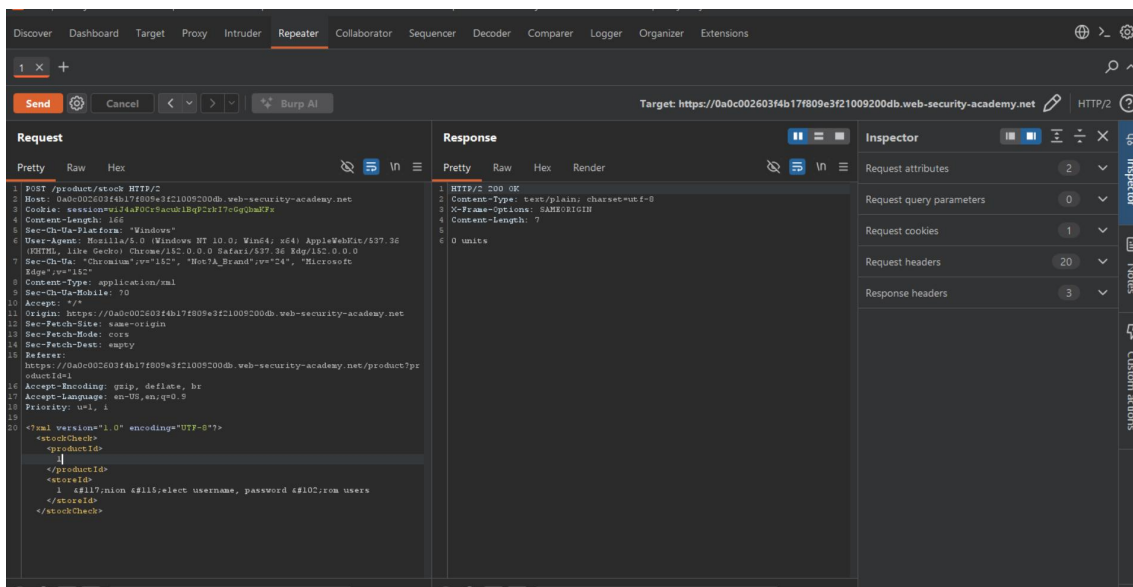
ALL SEARCH IMAGES VIDEOS MAPS COPILOT MORE

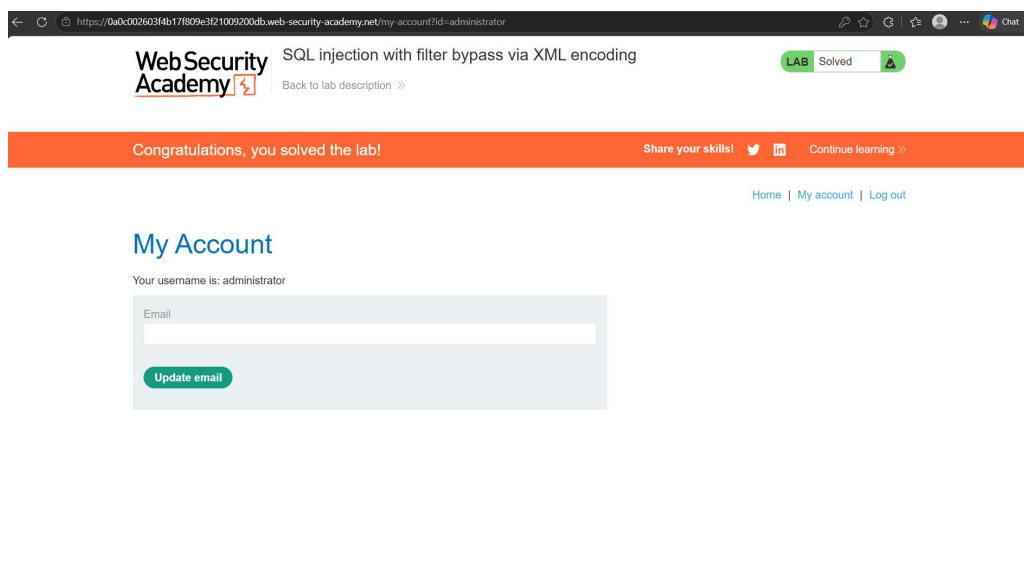
In **XML**, the letter **f** does **not** require escaping — only a small set of characters have special meaning and must be replaced with escape sequences:

Character	Escape Sequence
&	<code>&amp;</code>
<	<code>&lt;</code>
>	<code>&gt;</code>
"	<code>&quot;</code>
'	<code>&apos;</code>

If you still want to **encode** the letter **f** (for example, to represent it as a numeric or named entity), you can use:

- **Decimal numeric entity:** `f` (since ASCII code for **f** is 102)
- **Hexadecimal numeric entity:** `f`





Impact

An attacker can craft a malicious request containing the XML-encoded SQL injection payload and send it directly to the vulnerable `/product/stock` endpoint, bypassing the WAF's keyword filtering. Once executed, the injected query runs against the backend database, allowing the attacker to:

- Extract all usernames and passwords from the users table
- Hijack the administrator account and gain full control of the application
- Access or exfiltrate any other sensitive data stored in the database
- Bypass security controls (the WAF) that the application relies on for protection

This vulnerability affects the entire user base whose credentials are stored in the database, and is especially dangerous because it defeats a defense mechanism the application depends on, giving a false sense of security while remaining fully exploitable.

Severity

High: no user interaction is required. an attacker can send the malicious request directly to the endpoint and extract data on demand.

Key Takeaway

I learned how a WAF can be bypassed using XML entity encoding, and that pattern-matching filters aren't a real substitute for parameterized queries. It took a few tries to figure out why the encoded payload was returning 0 units even after slipping past the WAF turned out I'd changed the query structure by accident, not the encoding. Using Hackvertor to encode the whole clause at once instead of hand-typing entities made it much more reliable, and it was satisfying to see it click into place.

Rating

★★★★★ (5/5)